Inventory & Truck-Stock Management Platform

Technical Documentation — Architecture, Build & Run Guides, and Production-Readiness Report for the Back-End API, Web Front-End, Flutter Mobile App, and Qt Desktop App.

Back-End · Node/Express/Prisma

Front-End · Next.js

Mobile · Flutter

Desktop · Qt / C++

Repositories: felipeact/inventory-system-api · felipeact/InventoryQtApp
Generated: June 18, 2026 · **Version:** 1.0

Table of Contents

1. System Overview

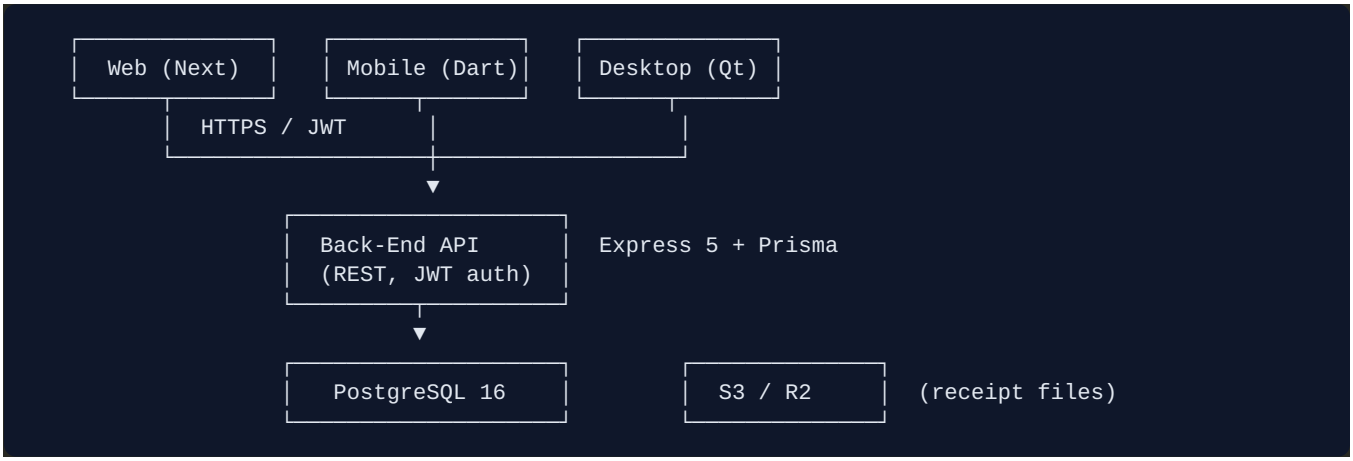
The **Inventory & Truck-Stock Management Platform** is a multi-tenant system that keeps a warehouse, its field technicians, and their service trucks in sync — real-time inventory, barcode scanning, truck-stock templates, receipt reconciliation, role-based access, and reporting. It is delivered as **four applications across two Git repositories**, all of which talk to a single shared back-end API.

1.1 Applications at a glance

#	Application	Repository / Path	Stack	Audience
1	**Back-End API**	`inventory-system-api/inventory_system_api`	Node.js 22, TypeScript, Express 5, Prisma 6, PostgreSQL 16	Server / all clients
2	**Web Front-End**	`inventory-system-api/web`	Next.js 15 (App Router), React 18, TypeScript, Tailwind	Admins / office
3	**Mobile App**	`InventoryQtApp/mobile`	Flutter 3, Dart, Provider, Dio	Field technicians
4	**Desktop App**	`InventoryQtApp/InventoryQtApp`	C++17, Qt 6.11, cpr, nlohmann/json	Warehouse / power users

1.2 Architecture

All clients are thin presentation layers over the same REST API. The API is the single source of truth and owns the database, authentication, multi-tenancy, and business rules.



- **Authentication** is stateless JWT access tokens (15 min) plus database-backed refresh tokens (7 days). Every client injects `Authorization: Bearer <token>` and transparently refreshes once on a `401`.
- **Multi-tenancy**: each user belongs to a `company`; the `companyId` from the token is threaded into every query so tenants are isolated.
- **RBAC**: roles (`ADMIN`, `WAREHOUSE`, `TECHNICIAN`) map to a permission set enforced per route.

1.3 Repository layout

```
inventory-system-api/                  InventoryQtApp/
├── inventory_system_api/   (API)      ├── InventoryQtApp/   (Qt desktop sources)
│   ├── src/                           │   ├── *.cpp / *.h / *.ui
│   ├── prisma/                        │   ├── CMakeLists.txt     (portable build)
│   ├── Dockerfile                     │   └── vcpkg.json         (pinned native deps)
│   └── docker-compose.yml             ├── mobile/           (Flutter app)
├── web/                    (Next.js)  │   ├── lib/ android/ ios/
│   └── src/app/                       │   └── pubspec.yaml
└── .github/workflows/ci.yml           ├── installer/        (Inno Setup .iss)
                                       └── deploy.ps1        (MSBuild + windeployqt)
```

2. Back-End API

Node.js / TypeScript / Express 5 REST API backed by PostgreSQL via Prisma.

2.1 Technology stack

Concern	Choice
Runtime	Node.js 22
Language	TypeScript 6 (`strict`)
Framework	Express 5
ORM / DB	Prisma 6 + `@prisma/adapter-pg` over PostgreSQL 16
Auth	JWT (`jsonwebtoken`) + bcrypt (cost 12)
Validation	Zod
Security	helmet, cors, express-rate-limit, compression
Logging	pino + pino-http (with secret redaction)
Files	AWS S3 / Cloudflare R2 or local filesystem
Reporting	pdfkit, xlsx
Tests	Jest + ts-jest + supertest
Deploy	Docker (multi-stage), docker-compose, Railway

2.2 Project structure

Feature-per-module under `src/modules/<feature>/` with `*.controller.ts / *.service.ts / *.repository.ts / *.routes.ts / *.validation.ts`.

```
src/
├── app.ts                Express factory (middleware + routes)
├── server.ts             HTTP listener + graceful shutdown
├── config/env.ts         Zod-validated environment configuration
├── lib/                  prisma.ts, logger.ts, storage.ts
├── core/                 app-error.ts, async-handler.ts, base.controller.ts
├── middleware/           auth, permission, super-admin, subscription, rate-limit, validate,
error
└── modules/              auth, user, products, asset, report, export, super-admin, truck-stock
```

2.3 Prerequisites

- Node.js 22 and npm
- PostgreSQL 16 (local install, Docker, or a managed instance)

2.4 Environment variables

Copy `.env.example` to `.env` and fill in values. The process **refuses to start** if a required variable is missing or (in production) weak.

Variable	Required	Notes
`DATABASE_URL`		PostgreSQL connection string
`JWT_SECRET`		`openssl rand -hex 32` ; must be strong in prod
`JWT_REFRESH_SECRET`		Distinct from `JWT_SECRET` in prod
`SUPER_ADMIN_BOOTSTRAP_SECRET`	—	Optional; gates the web/HTTP bootstrap of the first super-admin. If blank, that endpoint is disabled in prod and you use `npm run seed` . Strong (16+ chars) when set
`PORT`	—	Default `3000`
`NODE_ENV`	—	`development` | `test` | `production`
`CORS_ORIGINS`	—	Comma-separated allowed origins. **Must include the web app's origin**
`STORAGE_DRIVER`	—	`local` | `s3` (S3 vars required when `s3`)
`SMTP_*`	—	Optional; email flows become no-ops if unset (see §2.10)
`ADMIN_NOTIFICATION_EMAIL`	—	Recipient for sign-up / lead / super-admin notifications
`SEED_*`	—	Optional; values for the `npm run seed` bootstrap (see §2.9)

2.5 Install, build & run

```
cd inventory-system-api/inventory_system_api

# 1. Install dependencies (reproducible)
npm ci

# 2. Generate the Prisma client
npm run generate

# 3. Apply database migrations
npm run migrate:deploy        # production / CI (applies existing migrations)
# npm run migrate             # development (creates a new migration)

# 4a. Run in development (auto-reload)
npm run dev

# 4b. Or build and run the production bundle
npm run build                 # -> dist/
npm start                     # node dist/src/server.js
```

Useful checks:

```
npm run typecheck             # tsc --noEmit
npm test                      # jest --runInBand
npm run test:coverage         # jest with coverage
npm run studio                # Prisma Studio DB browser
```

2.6 Health & readiness

Endpoint	Purpose
`GET /health`	Liveness — process is up (no dependencies checked)
`GET /ready`	Readiness — verifies database connectivity (`503` when DB is down)

Use `/health` for container liveness (Docker/Railway) and `/ready` for load-balancer / Kubernetes readiness probes.

2.7 Running with Docker

```
# Single image
docker build -t inventory-api .
docker run --env-file .env -p 3000:3000 inventory-api

# Full stack (API + PostgreSQL) — applies migrations on boot
docker compose up --build
```

> The Docker image is multi-stage, runs as the non-root `node` user, and ships a `HEALTHCHECK`. Provide strong `JWT_SECRET`, `JWT_REFRESH_SECRET`, and `SUPER_ADMIN_BOOTSTRAP_SECRET` via the environment for any non-local run.

2.8 Deploying to Railway

`railway.json` builds the Dockerfile, runs `prisma migrate deploy`, then starts the server, with `healthcheckPath: /health`. Set the environment variables in the Railway dashboard.

2.9 First-run onboarding

A fresh database has no accounts, and **registration requires an activation code** — so the very first thing to do is mint one. There are three ways, easiest first.

Option A — Web operator console (recommended)

The web front-end ships a built-in super-admin console. No curl required.

1. Open `https://<your-web-app>/admin/setup`.
2. Enter an email, a password, and the `SUPER_ADMIN_BOOTSTRAP_SECRET` you set on the API. (This works only once; the endpoint closes after the first super-admin exists.)
3. You land on `/admin`. Click **Create an activation code** (a random code is pre-filled). Pick a plan and limits, and create it.
4. Go to `/register`, enter a company name, email, password, and that activation code. You now have a test account and are signed into the dashboard.

The console also lists every activation code and registered company, and lets you disable codes or activate/deactivate tenants.

Option B — Seed script (one command)

The repo ships an **idempotent** seed that creates the first super-admin *and* a sample activation code. Run it once after migrations (locally, or via a Railway one-off shell):

```
npm run seed
# ✔ Super-admin created: admin@inventory.local  (default password ChangeMe!2026)
# ✔ Activation code created: TEST-2026 (plan PRO, 25 users, 5000 products)
```

Override any value with `SEED_SUPER_ADMIN_EMAIL`, `SEED_SUPER_ADMIN_PASSWORD`, `SEED_ACTIVATION_CODE`, `SEED_ACTIVATION_PLAN`, `SEED_ACTIVATION_MAX_USERS`, `SEED_ACTIVATION_MAX_PRODUCTS`. Re-running never duplicates or overwrites existing rows. Then register at `/register` with the code (default `TEST-2026`).

Option C — Raw API (curl)

```
API=https://your-api-host

# 1. Create the first super-admin (requires the bootstrap secret header)
curl -X POST "$API/super-admin/create" \
  -H "Content-Type: application/json" \
  -H "x-bootstrap-secret: $SUPER_ADMIN_BOOTSTRAP_SECRET" \
  -d '{"email":"root@example.com","password":"a-strong-password"}'

# 2. Log in as super-admin to get a token
curl -X POST "$API/super-admin/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"root@example.com","password":"a-strong-password"}'

# 3. Create an activation code (with the super-admin token)
curl -X POST "$API/super-admin/activation-codes" \
  -H "Authorization: Bearer <SUPER_ADMIN_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"code":"WELCOME2026","plan":"PRO","maxUsers":25,"maxProducts":5000}'
```

In every case, registration creates the company, default roles (`ADMIN` / `WAREHOUSE` / `TECHNICIAN`), seeded permissions, and the first admin user, then returns access + refresh tokens.

2.10 Email & notifications

Transactional email and operational notifications are sent over SMTP. **Email is optional**: if SMTP is unconfigured, these flows log a warning and become no-ops — they never crash a request.

Trigger	Recipient	Email
Company registers	New admin user	Welcome email
Company registers	`ADMIN_NOTIFICATION_EMAIL`	"New company registered"
Super-admin created	`ADMIN_NOTIFICATION_EMAIL`	"Super-admin account created"
Demo/contact form (`POST /leads`)	`ADMIN_NOTIFICATION_EMAIL`	"New demo / contact request"
Password reset requested	The user	Reset link (30-min expiry)
User invited (admin)	The invitee	Temporary password

The provider is chosen automatically: if `RESEND_API_KEY` is set, email goes over Resend's HTTPS API; otherwise the `SMTP_*` settings are used. Sends are time-boxed (~10s) and never block the originating request.

> **Railway note:** many PaaS providers (Railway included) **block outbound SMTP ports**, which makes Gmail/SMTP hang or time out. If your emails don't arrive and the logs show an SMTP timeout, use **Resend** below — it sends over HTTPS (port 443) and is not affected.

Option 1 — Resend (recommended on Railway):

1. Create a free account at resend.com using `felipetiburcioviana@gmail.com`.
2. Create an **API key**.
3. Set on the API host (Railway → Variables):

```
RESEND_API_KEY=re_xxxxxxxx
RESEND_FROM=onboarding@resend.dev        # works in test mode without a verified domain
ADMIN_NOTIFICATION_EMAIL=felipetiburcioviana@gmail.com
FRONTEND_URL=https://<your-web-app>
```

In test mode (no verified domain) Resend delivers from the shared onboarding sender **to the address you signed up with** — exactly what's needed for owner notifications. To email arbitrary users (e.g. welcome emails), verify a domain in Resend and set `RESEND_FROM` to it.

Option 2 — Gmail SMTP (works only where outbound SMTP is allowed):

1. Enable **2-Step Verification** on the Google account.
2. Create an **App password** (Google Account → Security → App passwords) — a 16-char code.
3. Set on the API host:

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=felipetiburcioviana@gmail.com
SMTP_PASS=<16-char app password>      # NOT your normal Google password
SMTP_FROM=Inventory System <felipetiburcioviana@gmail.com>
ADMIN_NOTIFICATION_EMAIL=felipetiburcioviana@gmail.com
FRONTEND_URL=https://<your-web-app>   # used for reset-password links
```

Restart the API; sign-ups, leads, and super-admin creation now arrive in your inbox.

---

3. Web Front-End

Marketing site **and** the authenticated admin dashboard, built with Next.js 15 (App Router).

3.1 Technology stack

Concern	Choice
Framework	Next.js 15 (App Router) + React 18
Language	TypeScript 5 (`strict` , zero `any`)
Styling	Tailwind CSS 3
Data	Typed `fetch` wrapper (`src/lib/api.ts`) with auto token-refresh
Charts / UI	recharts, framer-motion, lucide-react

3.2 Routes

Group	Routes
Marketing	`/` , `/features` , `/pricing` , `/request-demo`
Auth	`/login` , `/register` , `/forgot-password`
App (guarded)	`/dashboard` , `/products` , `/assets` , `/trucks` , `/reports` , `/settings`
Operator (super-admin)	`/admin/setup` , `/admin/login` , `/admin` (activation codes + tenants)
Generated	`/robots.txt` , `/sitemap.xml` , `/icon.svg` , `/_not-found`

> The `/admin/*` console uses a separate super-admin token (stored under its own key, 12-hour expiry, `noindex`). It is how operators mint activation codes and manage tenants — see §2.9.

3.3 Prerequisites

- Node.js 22 and npm
- A running back-end API reachable from the browser

3.4 Environment variables

`NEXT_PUBLIC_*` values are inlined into the client bundle **at build time**.

Variable	Required	Notes
`NEXT_PUBLIC_API_BASE_URL`	(prod build fails without it)	API origin, no trailing slash
`NEXT_PUBLIC_SITE_URL`	—	Canonical site URL for OG/canonical/sitemap
`NEXT_PUBLIC_SALES_EMAIL`	—	Display-only sales contact

3.5 Install, build & run

```
cd inventory-system-api/web
cp .env.example .env.local          # then edit values

npm install                         # or: npm ci

# Development (API also defaults to :3000, so run web on another port)
npm run dev -- -p 3001

# Quality gates
npm run lint
npm run typecheck

# Production build & serve (API URL is required for the build)
NEXT_PUBLIC_API_BASE_URL=https://your-api-host npm run build
npm start
```

> Because the front-end and API both default to port `3000`, run the web app on `3001` during local development **and** add `http://localhost:3001` to the API's `CORS_ORIGINS`.

3.6 Deployment

- **Vercel**: zero-config. Set `NEXT_PUBLIC_API_BASE_URL` (and `NEXT_PUBLIC_SITE_URL`) in project settings.
- **Docker / Node container**: the config sets `output: "standalone"`, so the build emits a self-contained server:

```
NEXT_PUBLIC_API_BASE_URL=https://your-api-host npm run build
node .next/standalone/server.js     # serves the app
```

3.7 Security headers

The app sets a Content-Security-Policy, `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, and `Permissions-Policy`, and disables the `X-Powered-By` header (see `next.config.mjs`).

---

4. Mobile App

Flutter app for field technicians — truck stock, scanning, receipts, and assignments.

4.1 Technology stack

Concern	Choice
Framework	Flutter 3 / Dart 3
State	Provider (ChangeNotifier)
HTTP	Dio (singleton client with 401 refresh)
Security	`flutter_secure_storage` for tokens
Features	`mobile_scanner` (barcode), `image_picker` (receipts), `fl_chart`

4.2 Project structure

```
mobile/
├── lib/
│   ├── main.dart            entrypoint (guarded zone + global error handling)
│   ├── app.dart             MultiProvider + MaterialApp
│   ├── config/              app_config.dart (API base URL), routes, theme
│   ├── models/ services/ providers/ widgets/ screens/   (18 screens)
├── android/                 Gradle project (release signing + R8)
├── ios/                     iOS runner (scaffold via `flutter create .`)
└── pubspec.yaml
```

4.3 Prerequisites

- Flutter SDK 3.x (run `flutter doctor` and resolve all checks)
- Android: Android Studio + SDK (compileSdk 34), a device/emulator
- iOS (optional): macOS + Xcode + CocoaPods

4.4 Configuration

The API base URL is a **runtime** value: it defaults to the production URL in `lib/config/app_config.dart` and can be overridden in the in-app **Settings** screen (persisted via `SharedPreferences`). The default is HTTPS-qualified, and any value entered without a scheme is normalised to `https://`.

4.5 Install & run (development)

```
cd InventoryQtApp/mobile

flutter pub get

# First time only: regenerate native scaffolding (gradlew, Runner.xcodeproj, Podfile…)
flutter create .

# Generate launcher icons (writes android mipmaps + iOS icons)
dart run flutter_launcher_icons

# Run on a connected device / emulator
flutter run
```

4.6 Android release build (Play Store)

1. Create an upload keystore (once) and a `key.properties` file:

```
keytool -genkey -v -keystore upload-keystore.jks \
  -keyalg RSA -keysize 2048 -validity 10000 -alias upload

# copy the template and fill in your values (NEVER commit key.properties or the .jks)
cp android/key.properties.example android/key.properties
```

`android/key.properties`:

```
storePassword=********
keyPassword=********
keyAlias=upload
storeFile=upload-keystore.jks
```

2. Build (R8 shrinking + your release signing are applied automatically when `key.properties` is present; otherwise it falls back to debug signing):

```
flutter build apk --release                       # APK for sideloading
flutter build appbundle --release                 # AAB for the Play Store

# Set the version explicitly for store uploads (recommended in CI):
flutter build appbundle --release --build-name=1.2.0 --build-number=5
```

Outputs: `build/app/outputs/flutter-apk/app-release.apk` and `build/app/outputs/bundle/release/app-release.aab`.

4.7 iOS release build

```
flutter create .                      # ensure ios/Runner.xcodeproj + Podfile exist
cd ios && pod install && cd ..
# In Xcode: set the Bundle Identifier, Team, and signing for Runner.
flutter build ipa --release           # produces build/ios/ipa/*.ipa
```

4.8 Network security

`android/app/src/main/res/xml/network_security_config.xml` rejects cleartext (HTTP) traffic in production and permits it only for `localhost`, `127.0.0.1`, and the emulator host `10.0.2.2` during development.

5. Desktop App

Qt 6 / C++ Windows desktop client for warehouse and power users, with an Inno Setup installer and an auto-updater.

5.1 Technology stack

Concern	Choice
Language	C++17
Framework	Qt 6.11 (Core, Gui, Widgets, Network)
HTTP / JSON	cpr (libcurl) + nlohmann/json
Excel export	QXlsx (optional, compile-time)
Build	Visual Studio 2022 (`.vcxproj`) **or** CMake + vcpkg
Packaging	Inno Setup (`installer/InventoryQtApp.iss`)

5.2 Project structure

```
InventoryQtApp/InventoryQtApp/
├── main.cpp                 entrypoint (app identity, file logger, version, crash guard)
├── Version.h                single source of truth for the app version
├── InventoryQtApp.*         login window + app shell
├── DashboardWindow.*        main window (QStackedWidget of pages)
├── *Page.* / *Dialog.*      ~19 pages and ~20 dialogs
├── ApiClient.*              HTTP client (bearer auth, single-flight 401 refresh)
├── *Service.*               thin API wrappers (Auth, Product, Asset, User, Report, TruckStock)
├── Config.h SecureStore.*   URL config + DPAPI-encrypted token storage
├── AutoUpdateManager.*      update check / download / launch
├── CMakeLists.txt           portable build (new)
└── vcpkg.json               pinned native dependencies (new)
```

5.3 Prerequisites

- Windows 10/11 x64
- Visual Studio 2022 with the **Qt VS Tools** extension
- **Qt 6.11.0 (msvc2022_64)** installed and registered
- **vcpkg** providing `cpr` , `nlohmann-json` , and (optional) `qxlsx`
- **Inno Setup** (`ISCC.exe`) for building the installer

5.4 Configuration

API and update-server URLs resolve in this order: environment variable → `QSettings` → built-in production default (`Config.h`).

Setting	Environment variable	QSettings key
API base URL	`INVENTORY_APP_API_URL`	`api/baseUrl`
Update URL	`INVENTORY_APP_UPDATE_URL`	`updates/checkUrl`

5.5 Build option A — Visual Studio / MSBuild

```
# From the repo root (InventoryQtApp/). Requires VS2022 + Qt VS Tools + registered Qt.
msbuild InventoryQtApp.slnx /p:Configuration=Release /p:Platform=x64 /m

# Or use the all-in-one script: build, stage, and run windeployqt
.\deploy.ps1 -Version 1.0.0 -QtDir "C:\Qt\6.11.0\msvc2022_64"
```

`deploy.ps1` locates MSBuild via `vswhere`, builds Release|x64, stages the executable, runs `windeployqt` to copy the Qt runtime, and copies the cpr/curl/zlib DLLs.

5.6 Build option B — CMake + vcpkg (portable / CI)

A `CMakeLists.txt` and `vcpkg.json` were added so the app can be built reproducibly without the Visual Studio project (useful for CI and non-VS contributors).

```
# Native dependencies come from the vcpkg manifest (vcpkg.json) automatically.
cmake -S InventoryQtApp -B build `
  -DCMAKE_TOOLCHAIN_FILE="$env:VCPKG_ROOT/scripts/buildsystems/vcpkg.cmake" `
  -DCMAKE_PREFIX_PATH="C:/Qt/6.11.0/msvc2022_64" `
  -DCMAKE_BUILD_TYPE=Release

cmake --build build --config Release
```

The CMake build enables AUTOMOC/AUTOUIC/AUTORCC, pins C++17, links Qt6 + cpr + nlohmann/json, auto-detects QXlsx, and runs `windeployqt` after build on Windows.

5.7 Building the installer

```
# Inno Setup compiler; pass the version to stamp the installer.
ISCC.exe /DAppVersion=1.0.0 installer\InventoryQtApp.iss
```

The version in `Version.h`, the `deploy.ps1 -Version`, and the `ISCC /DAppVersion` should all match. `main.cpp` writes `Version.h`'s value into the `app/version` setting that the auto-updater compares against the server's latest release.

5.8 Logs

The desktop app installs a Qt message handler that writes a rolling log file to the per-user app-data location (e.g. `%APPDATA%/InventorySystem/InventoryQtApp/inventory-app.log`).

6. Production-Readiness Report

This section summarises the audit performed on all four apps and the hardening applied in this pass.

6.1 Methodology

Each application was audited for: security, configuration/secrets, error handling, logging, health/readiness, testing, CI/CD, performance, and build reproducibility. Findings were ranked **Critical** (security / data-loss), **High** (reliability), **Medium** (best practice), and **Low** (polish).

6.2 What was fixed in this pass

Back-End *(verified: typecheck, 26 tests, production build all pass)*

Area	Change
Super-admin takeover	`POST /super-admin/create` now requires a timing-safe `x-bootstrap-secret`; the secret is mandatory in production
Brute force	Added the strict auth rate-limiter to `/super-admin/create` and `/super-admin/login`
Session safety	Password change / reset / admin-reset now revoke all refresh tokens
Weak temp password	Admin reset now uses `crypto.randomBytes` instead of `Math.random()`
Password hashing	bcrypt cost raised from 10 to 12 everywhere
Readiness	Added `GET /ready` that verifies database connectivity (503 on failure)
Forced password change	`mustChangePassword` is now enforced server-side (allowlisting only the change-password/ validate endpoints)
Error contract	Missing product returns `404` (was `500`); added a JSON 404 handler for unknown routes
Resilience	Added an `uncaughtException` handler with graceful shutdown
Log hygiene	pino now redacts auth headers, cookies, secrets, passwords, and tokens
Performance	Added response `compression`
CI	Added a Web (Next.js) lint + typecheck + build job

Web Front-End *(verified: lint, typecheck, production build all pass)*

Area	Change
Security headers	Added CSP, HSTS, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`; disabled `X-Powered-By`
Error UX	Added `error.tsx`, `global-error.tsx`, `not-found.tsx`, and `loading.tsx`
Config safety	Production build now fails if `NEXT_PUBLIC_API_BASE_URL` is unset (no silent localhost)
Auth	`downloadExport` now uses the shared 401-refresh-and-retry path
Hardening	`images.remotePatterns` restricted to the API host; `output: standalone` for slim deploys
SEO	Added `robots.ts`, `sitemap.ts`, a favicon/ `icon.svg`, env-driven `metadataBase`, and home-page metadata
Tooling	Added a `typecheck` script

Mobile App *(static fixes; build a release on-device to verify)*

Area	Change
Launch blocker	Default API URL is now HTTPS scheme-qualified (was a bare host → all calls failed); URL handling self-heals legacy values
Crash visibility	Added a guarded zone with `FlutterError.onError` and `PlatformDispatcher.onError`
Release signing	`build.gradle` reads an upload keystore from `key.properties` (template added); falls back to debug when absent
Shrinking	Enabled R8 `minifyEnabled` + `shrinkResources` with `proguard-rules.pro` keep rules
Network	Added a network-security-config that blocks cleartext in production
Polish	App label moved to a string resource

Desktop App *(static fixes; build with Qt/MSVC to verify)*

Area	Change
Wrong endpoint	Replaced the hardcoded ephemeral Codespaces URL with the production API/update URL
Versioning	Added `Version.h` as the single source of truth; `main.cpp` publishes it to the `app/version` setting the updater reads
Logging	Added a Qt message handler that writes a rolling per-user log file
Identity / crashes	Set organization/app/version identity and added a top-level exception guard in `main.cpp`
Reproducible builds	Added `CMakeLists.txt` (portable/CI) and `vcpkg.json` (pinned native deps)
Hygiene	`.gitignore` now excludes build/deploy/installer outputs and signing material

6.3 Remaining recommendations

These require your secrets, hardware, or toolchains, or are larger changes best done with full QA. They are documented here as the next steps toward a hardened release.

Back-End — Add service-level / RBAC / tenant-isolation tests with coverage thresholds; add Zod schemas to the `/truck-stock` routes; pin the Node base image by digest; note that `xlsx@0.18.5` carries registry advisories (track the vendor build). *(A deterministic Prisma seed for the first super-admin + sample activation code now ships — see §2.9.)*

Web — Move JWTs (especially the refresh token) out of `localStorage` into `httpOnly` cookies via a small BFF/route-handler to remove the XSS token-theft vector, then enforce auth in `middleware.ts` (server-side) instead of the client-only guard.

Mobile — Generate and commit launcher icons (`dart run flutter_launcher_icons`); create and configure the iOS Xcode project + signing; integrate crash reporting (Crashlytics/Sentry) into the new error sink; replace the role-name permission heuristic with the server's real permission list; **verify a signed release build on a device** (R8 is now enabled).

Desktop — Verify the downloaded auto-update binary with a SHA-256 (served in the update manifest) and an Authenticode signature **before** launching it (currently only size is checked — a remaining RCE risk); code-sign the installer and the executable for SmartScreen/UAC trust; reconcile or delete the stale root `InventoryQtApp.vcxproj` .

6.4 Account onboarding & notifications (this pass)

Area	Change
Self-serve onboarding	Added a web **super-admin console** (`/admin/setup` , `/admin/login` , `/admin`) to bootstrap the first operator, mint/disable activation codes, and activate/deactivate tenants — no curl required
Seed	Added an **idempotent** `npm run seed` that creates the first super-admin + a sample activation code (env-overridable; safe to re-run)
Welcome email	Registration now sends a welcome email to the new admin (previously dead code)
Owner notifications	New emails to `ADMIN_NOTIFICATION_EMAIL` on company sign-up, super-admin creation, and demo/lead submissions
Lead capture	Added a public, rate-limited `POST /leads` endpoint; the marketing **request-a-demo** form now posts to it (was a simulated no-op) and emails the owner
Email hardening	All email bodies HTML-escape user input; SMTP failures are caught/logged so they never break a request; consistent branded template
Web config	`NEXT_PUBLIC_API_BASE_URL` example points at the live API; documented the matching `CORS_ORIGINS` requirement

Verification: back-end `typecheck` + 26 tests pass; web `lint` + `typecheck` + production `build` pass.

7. Quick Command Reference

```
# ── Back-End API ──
cd inventory-system-api/inventory_system_api
npm ci && npm run generate && npm run migrate:deploy
npm run seed                                # bootstrap first super-admin + sample code
npm run dev                                 # development
npm run build && npm start                  # production
npm run typecheck && npm test               # checks
docker compose up --build                   # API + PostgreSQL

# ── Web Front-End ──
cd inventory-system-api/web
npm install
npm run dev -- -p 3001                       # development
npm run lint && npm run typecheck
NEXT_PUBLIC_API_BASE_URL=https://api npm run build && npm start

# ── Mobile (Flutter) ──
cd InventoryQtApp/mobile
flutter pub get && flutter create .
dart run flutter_launcher_icons
flutter run                                  # development
flutter build appbundle --release            # Play Store AAB

# ── Desktop (Qt) ──
cd InventoryQtApp
.\deploy.ps1 -Version 1.0.0                   # MSBuild + windeployqt
#   …or CMake:
cmake -S InventoryQtApp -B build -DCMAKE_TOOLCHAIN_FILE=<vcpkg.cmake> -DCMAKE_PREFIX_PATH=<Qt>
cmake --build build --config Release
ISCC.exe /DAppVersion=1.0.0 installer\InventoryQtApp.iss
```